

EEE8089

M2M Technology IoT

Topic 5: Closed Loop Control Implementation

Dr. Bystrov

School of Engineering
University of Newcastle upon Tyne

Aims/Objectives

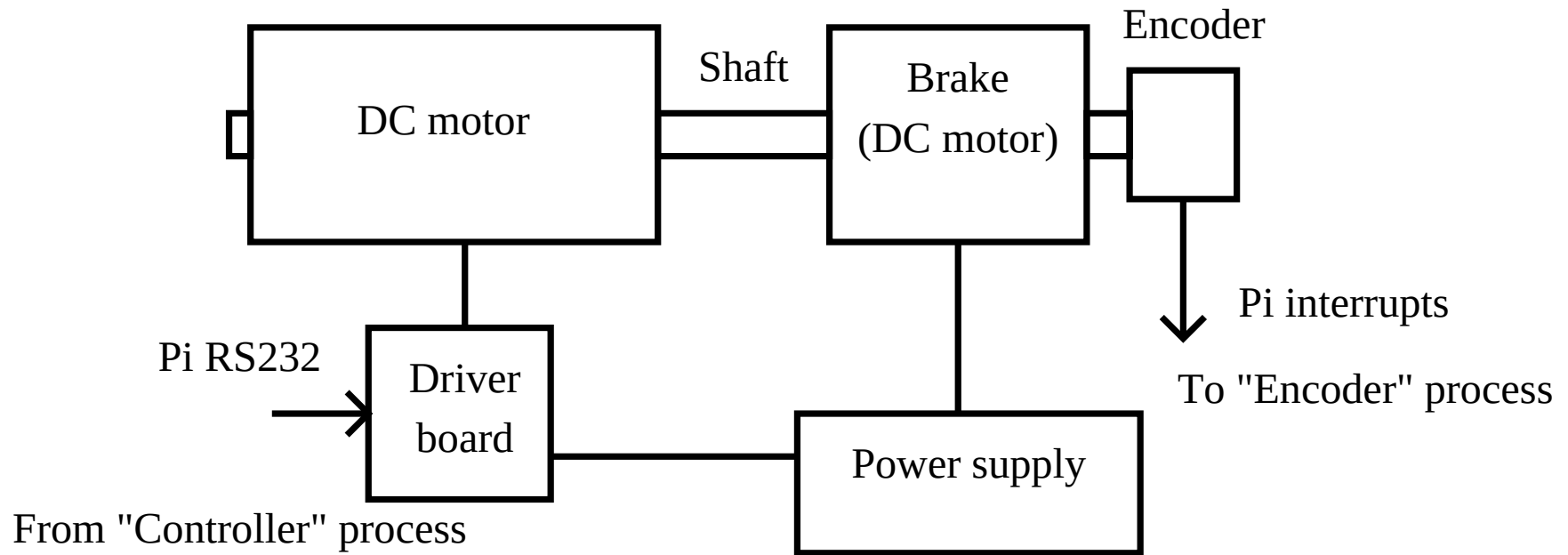
Aim

- The structure of the software implementation of the closed loop control system

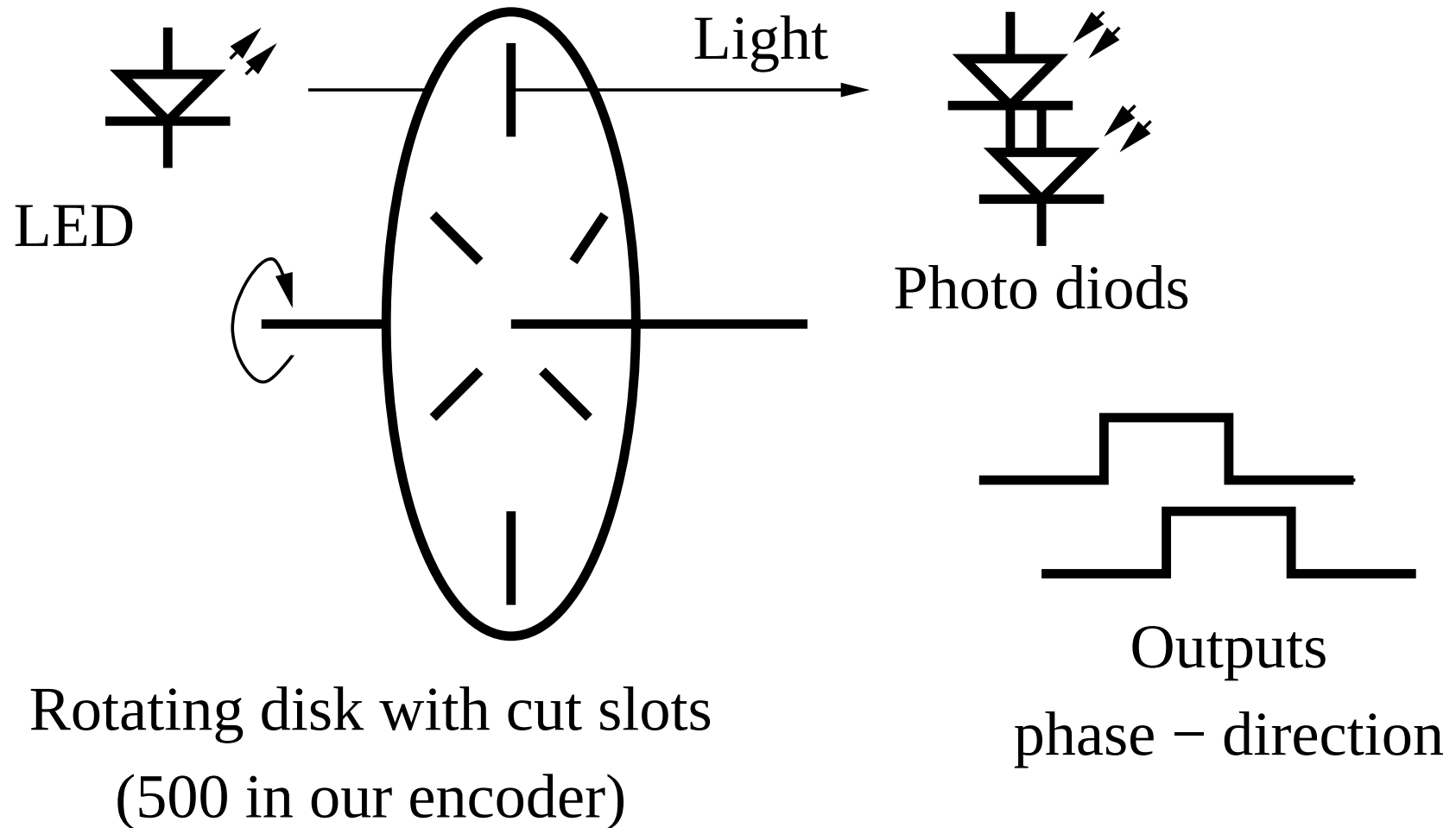
Objectives

- ...architecture review/reminder
- Encoder driver algorithm
- Controller algorithm
- Parameters for the controller

Control system – the rig



Control system – encoder



Control system – driver board

- RS232 duplex 19200 bps
- Input format: “%d %d\\”
 - motor input: 737 – off, +-500 left-right direction
 - enable/disable 1/0
- /dev/serial0
 - “echo 800 1\\ > /dev/serial0”
 - “cat /dev/serial0”
- RS232 has to be enabled in the system settings of Pi
- Power supply to the board +-20V

Control system – interrupts for encoder

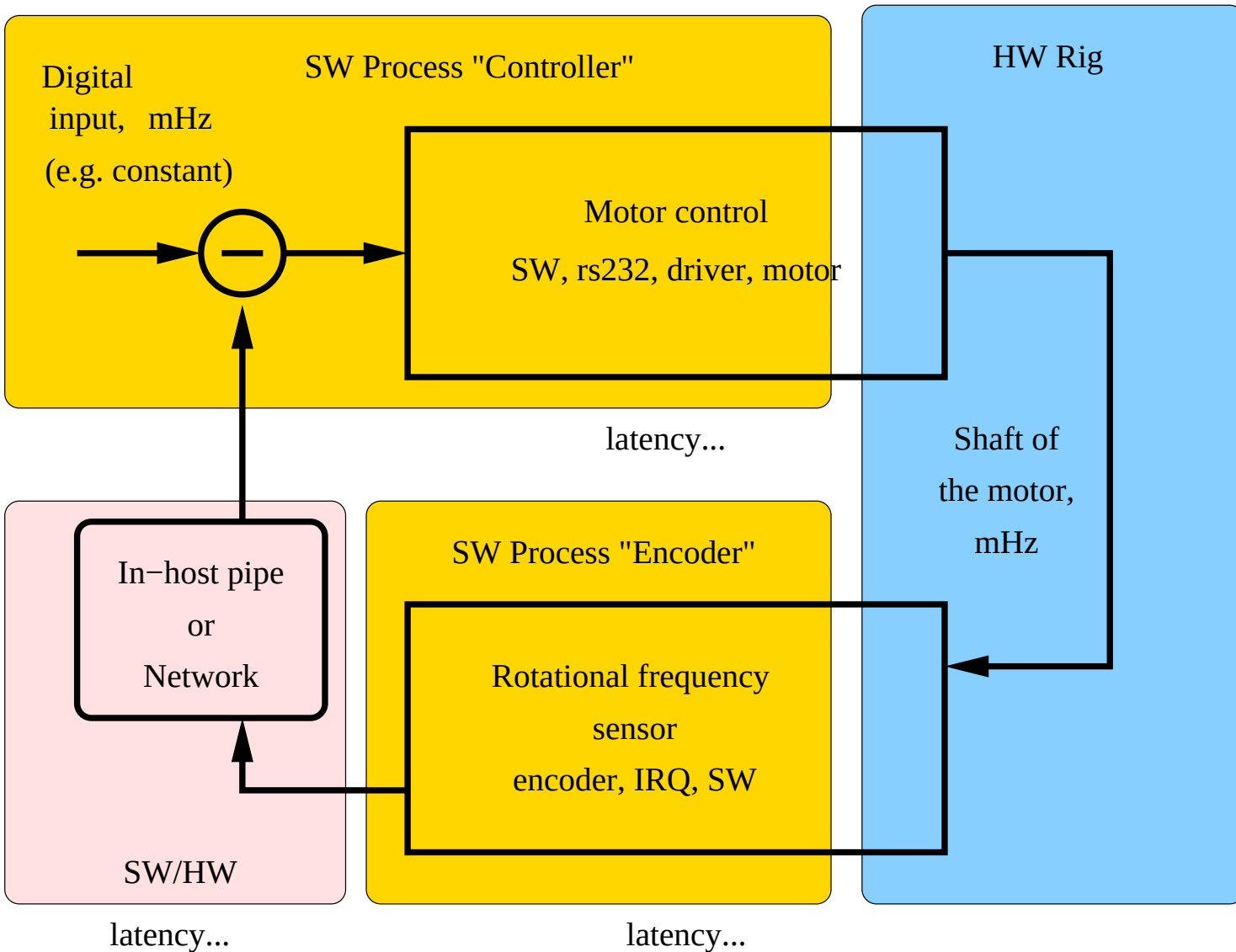
To configure e.g. IRQ9 on the pin GPIO9:

- `echo 9 > /sys/class/gpio/export`
- `echo in > /sys/class/gpio/gpio9/direction`
- `echo rising > /sys/class/gpio/gpio9/edge`

To access the interrupt input:

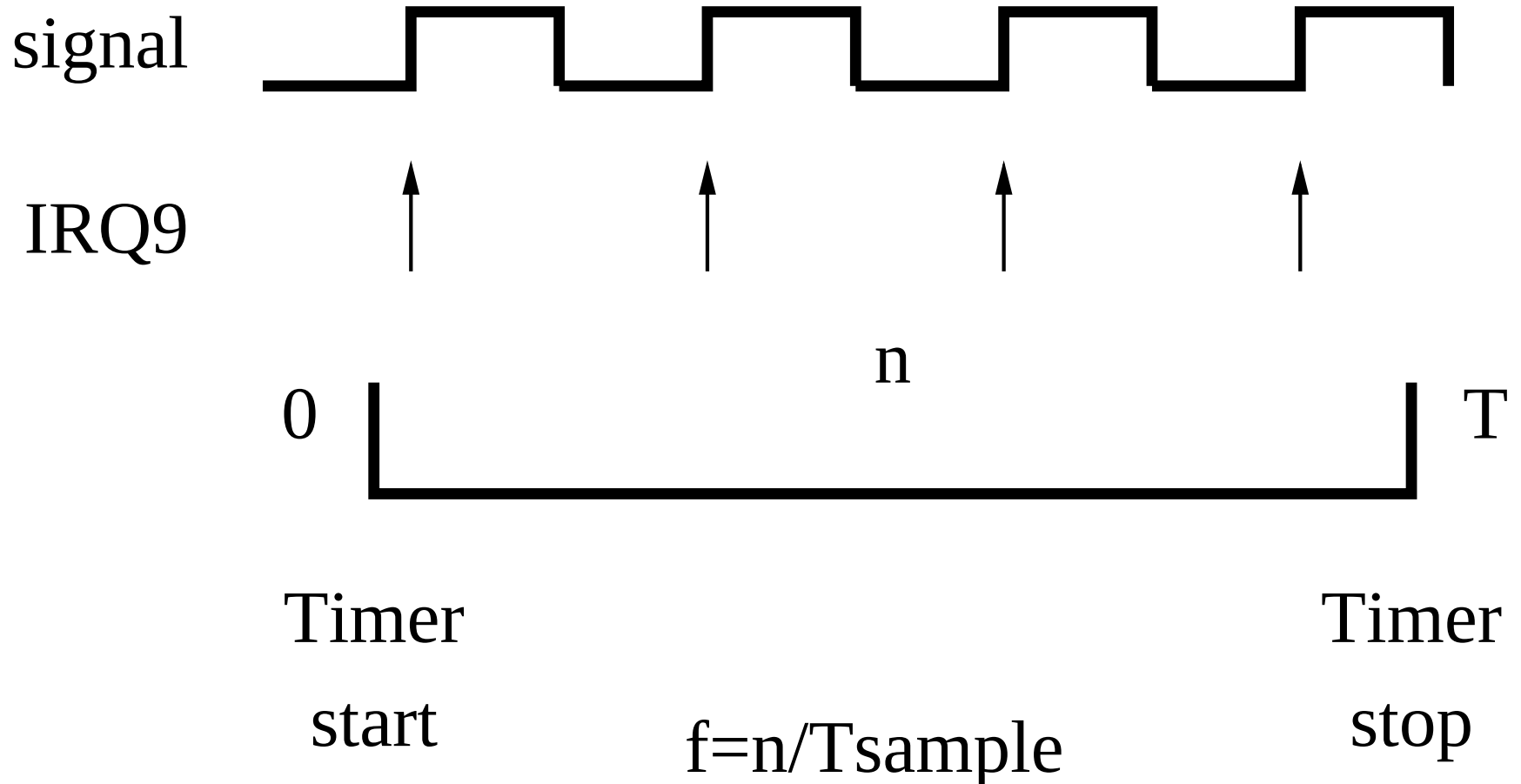
- use the standard library poll, header `<poll.h>`
- use function `poll()`, which freezes and then is released by the interrupt
- read “man poll”
- example of use is provided on Blackboard

Control system model



Frequency measurement

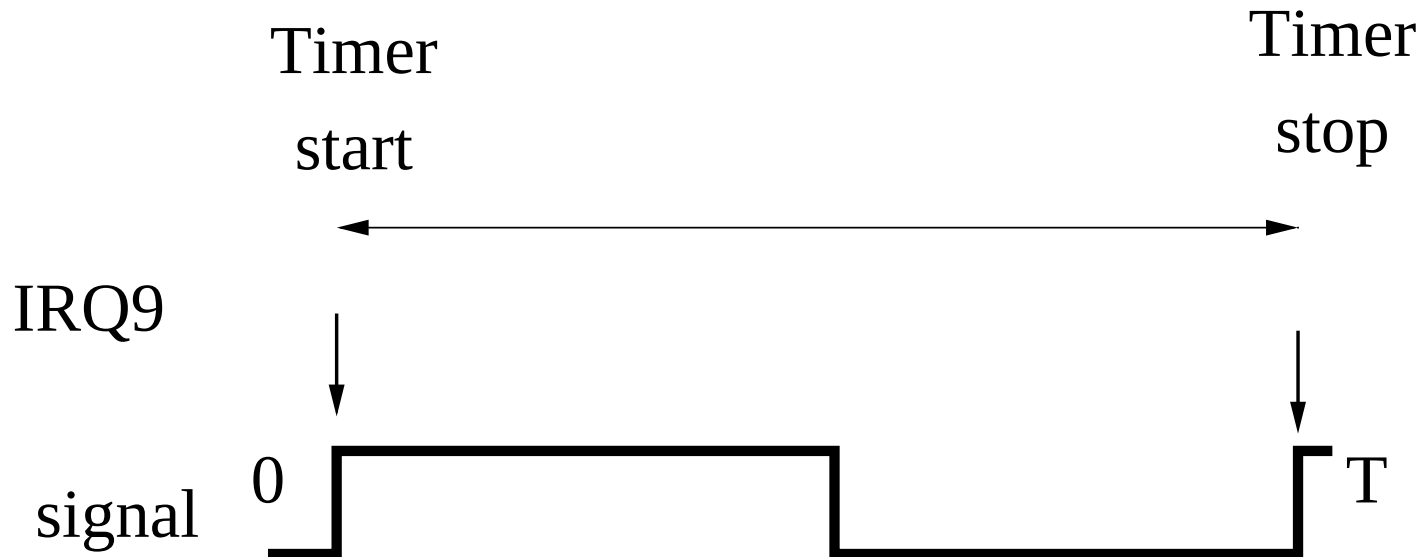
- Frequency is measured by counting interrupts within a set interval of time



- +/- 10 rotations per second, 500 pulses per rotation

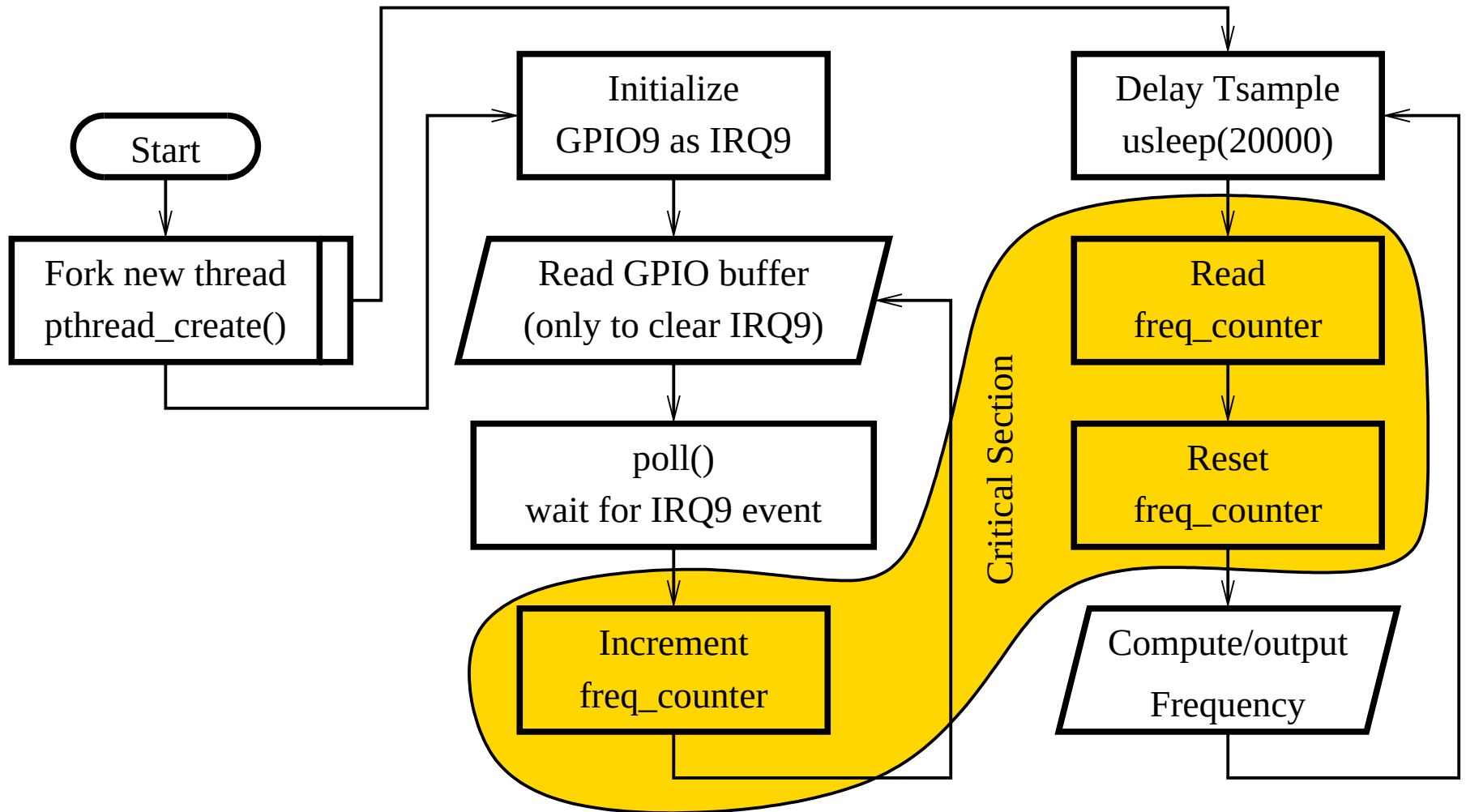
Period measurement

- For very slow rotational speeds one may use the period measurement



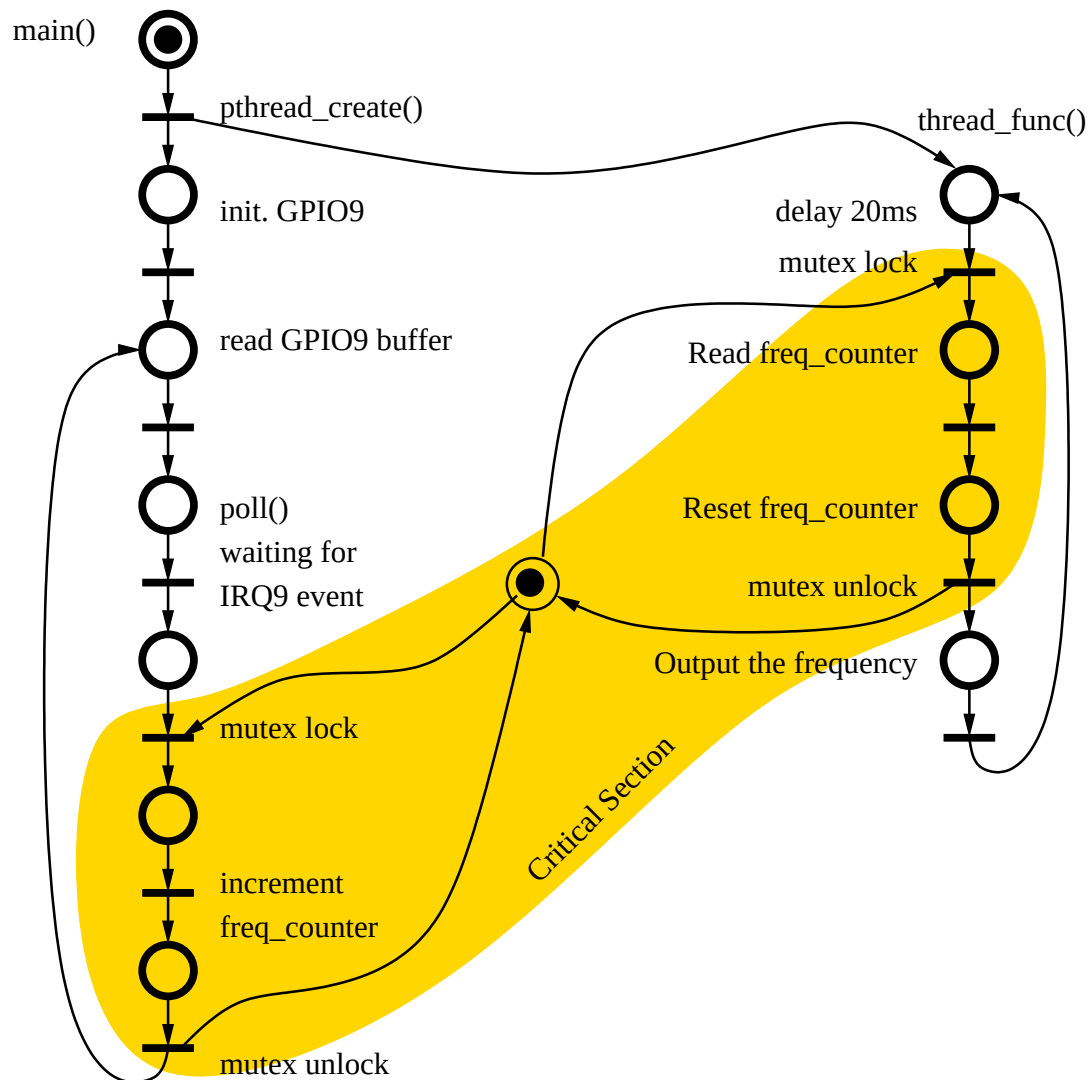
- If the accuracy of the interrupt is about $50\mu s$, then at about $5ms$ the measurement will become reasonably accurate at about 1%. The frequency measurement method will achieve a similar accuracy only after accumulating 100 pulses at 5ms period, i.e. $0.5s$.

Encoder driver algorithm



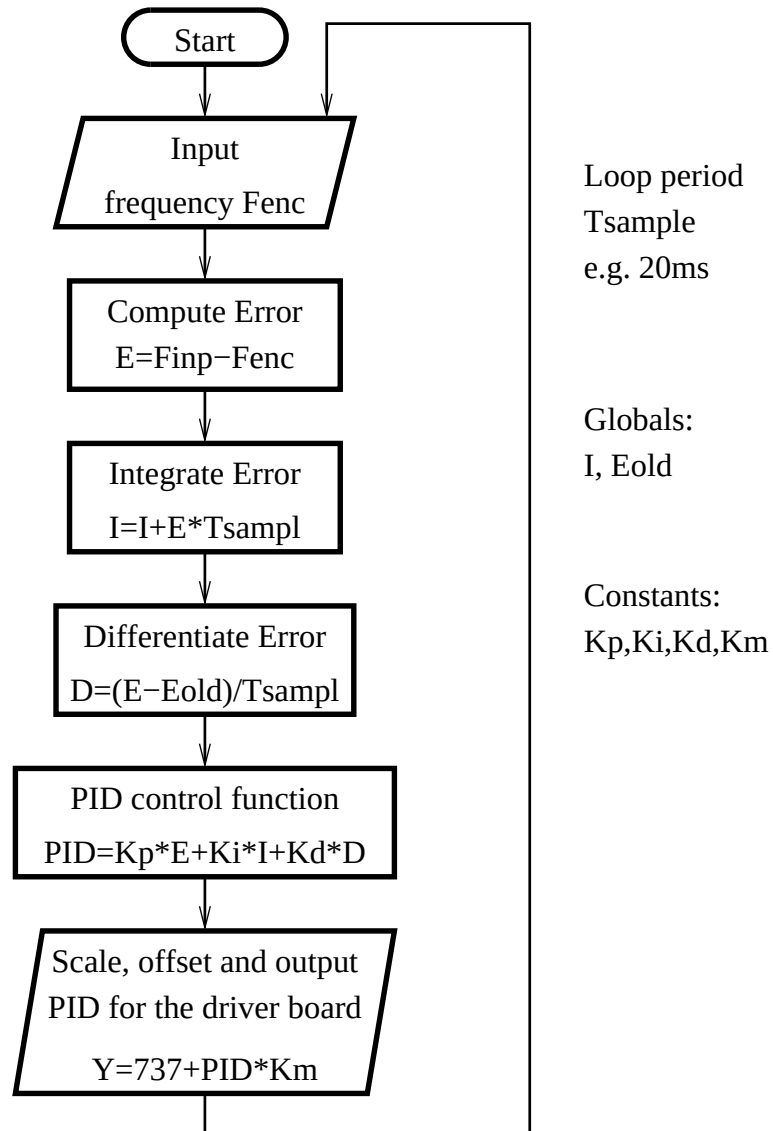
● See EEE8087 examples on threads and mutexes

Critical section on freq_counter



Resource access conflict on freq_counter – use mutex!

Controller algorithm



Controller – the constants

- K_p , K_i , and K_d are the constants of the PID controller model
 - Modelling in Maxima or Matlab;
 - I used approximate values — adjust them.
- K_m is a constant that translates the input to the driver board into the rotational speed of the motor
 - must be measured;
 - if the open-loop motor response is non-linear (e.g. a dead zone in the middle), then consider linearisation. In such a case Y will be computed in a more complex way (discussion...)

Revision of the control model



<https://www.staff.ncl.ac.uk/a.bystrov/eee8089/eee8089.html>



The time constant of the motor is assumed to be 10 units and the loop delay is 1 unit

- it is known that the electronics is faster than the motor, but by how much faster?
- in our case the encoder sampling period is 20ms, hence latency is 10 ms, the motor time constant is about 100ms, which is 1/10 ratio as in the simulation tutorial. However, the different sets of equipment may differ and the sampling rate can be increased to improve accuracy... Then one may need to repeat the simulations for a different set of parameters.



How to measure these values on our rig?

Calibrate the output to the driver board

- Use the Raspberry Pi to output the duty cycle values to the driver board of the motor
 - Format: “X 1\\”, where X is an integer in the range [737, 737+500]
 - Range: [737-500, 737+500], where 737 is “zero” speed
- Experimentally determine the value X for the rotational speed 1Hz – this is our no-load open-loop mode.
 - Calculate the coefficient converting the input of the driver board into the revolutions of the motor as $\frac{1}{X-737}$
 - Take the note of the offset 737, which has to be added to the output of the controller program.

Latency of the control loop

The latency includes the delay in the following components:

- Encoder driver – 50% of the sampling period, which is 20ms or another value of your choice
- RS232 interface at 19200bps, 8 bytes with start-stop bits, which gives about 4.2ms
- Driver board, under 0.5ms
- Pipelines of Linux, under 0.1ms

So, the latency is dominated by the encoder driver.

The time constant of the motor

- Option 1:** Use the encoder to measure the step response of the motor, but the latency in the encoder driver has to be taken into account (the sampling period)
- Option 2:** Use the oscilloscope connected to the other motor (acting as a generator) to measure the step response.
- Option 3:** Construct a simple control loop with proportional only control and increase K_p to the point of oscillation. Then use the Bode diagrams from the modelling and simulation tutorial to determine the cut-off frequency of the motor. Adjust the simulations if different.

Conclusions

- Modelling of the controller is reviewed
 - components
 - open-closed loop
 - step function (Heaviside) response
 - P, PI, PID controllers
- Encoder driver algorithm
- Controller algorithm
- Measurement of the component parameters for the controller design